

# main-backend

Python Documentation

Files: 22 | Extensions: .py

---

# Project Structure

```
main-backend/
├── src/
│   ├── main_backend/
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── health.py
│   │   │   │   └── responses.py
│   │   │   └── app/
│   │   │       ├── middleware/
│   │   │       │   ├── exception_handler.py
│   │   │       │   └── request_id.py
│   │   │       ├── predictions/
│   │   │       │   ├── router.py
│   │   │       │   ├── schemas.py
│   │   │       │   └── service.py
│   │   │       └── dependencies.py
│   │   ├── core/
│   │   │   ├── constants.py
│   │   │   ├── exceptions.py
│   │   │   ├── protocols.py
│   │   │   └── security.py
│   │   ├── db/
│   │   ├── domain/
│   │   │   └── entities.py
│   │   ├── external/
│   │   │   ├── database/
│   │   │   │   ├── config.py
│   │   │   │   ├── models.py
│   │   │   │   ├── repositories.py
│   │   │   │   └── unit_of_work.py
│   │   │   └── integrations/
│   │   │       ├── auth_client.py
│   │   │       └── forecasting_client.py
│   │   ├── config.py
│   │   ├── dependencies.py
│   │   └── main.py
│   ├── .dockerignore
│   ├── .env
│   ├── .python-version
│   ├── alembic.ini
│   ├── Dockerfile
│   ├── main-backend_python_docs.pdf
│   ├── pyproject.toml
│   └── README.md
```

# Source Code (Python)

## src/main\_backend/api/responses.py

```
1 | # src/main_backend/api/responses.py
2 | from datetime import datetime, timezone
3 | from typing import Generic, Optional, TypeVar
4 |
5 | from pydantic import BaseModel
6 |
7 | T = TypeVar("T")
8 |
9 |
10 | class ApiResponse(BaseModel, Generic[T]):
11 |     """Стандартная обёртка ответа API."""
12 |     status: str = "ok"
13 |     data: Optional[T] = None
14 |     meta: Optional[dict] = None
15 |     timestamp: str
16 |
17 |     def __init__(self, **data):
18 |         if "timestamp" not in data:
19 |             data["timestamp"] = datetime.now(timezone.utc).isoformat()
20 |         super().__init__(**data)
21 |
22 |
23 | class ErrorDetail(BaseModel):
24 |     """Детали ошибки."""
25 |     code: str
26 |     message: str
27 |     details: Optional[dict] = None
28 |
29 |
30 | class ErrorResponse(BaseModel):
31 |     """Стандартный ответ с ошибкой."""
32 |     status: str = "error"
33 |     error: ErrorDetail
34 |     timestamp: str
35 |
36 |     def __init__(self, **data):
37 |         if "timestamp" not in data:
38 |             data["timestamp"] = datetime.now(timezone.utc).isoformat()
39 |         super().__init__(**data)
```

## src/main\_backend/api/v1/health.py

```
1 | from __future__ import annotations
2 |
3 | import logging
4 | from typing import Annotated
5 |
6 | from fastapi import APIRouter, Depends, HTTPException, status
7 |
8 | from main_backend.core.security import (
9 |     CurrentUser,
10 |     require_admin,
```

## src/main\_backend/api/v1/health.py (continued)

```

11 |     verify_access_token,
12 | )
13 | from ...dependencies import get_auth_client
14 | from ...integrations.auth_client import AuthClient, UserNotFoundError, AuthClientError
15 |
16 | logger = logging.getLogger(__name__)
17 |
18 | router = APIRouter(prefix="/api/v1/health", tags=["Health"])
19 |
20 |
21 | @router.get("/whoami")
22 | async def whoami(
23 |     current_user: Annotated[CurrentUser, Depends(verify_access_token)],
24 | ):
25 |     """
26 |     Тестовый endpoint: возвращает данные ТОЛЬКО из JWT токена.
27 |
28 |     - Не делает HTTP запросов в auth-service
29 |     - Быстрый ответ (~5ms)
30 |     - Полезно для проверки JWT валидации
31 |     """
32 |     return {
33 |         "status": "ok",
34 |         "source": "jwt_only",
35 |         "user": {
36 |             "sub": str(current_user.sub),
37 |             "role": current_user.role,
38 |             "jti": str(current_user.jti),
39 |             "exp": current_user.exp.isoformat(),
40 |             "iat": current_user.iat.isoformat(),
41 |         },
42 |     }
43 |
44 |
45 | @router.get("/me")
46 | async def get_current_user_profile(
47 |     current_user: Annotated[CurrentUser, Depends(verify_access_token)],
48 |     auth_client: Annotated[AuthClient, Depends(get_auth_client)],
49 | ):
50 |     """
51 |     Полная информация о текущем пользователе.
52 |
53 |     Flow:
54 |     1. Валидирует JWT токен (локально, быстро)
55 |     2. Делает HTTP запрос в auth-service для получения полных данных
56 |     3. Возвращает объединённую информацию
57 |
58 |     - Медленнее чем /whoami (50-200ms из-за HTTP запроса)
59 |     - Возвращает email, username, phone и другие поля
60 |     - Проверяет, что пользователь активен
61 |     """
62 |     try:
63 |         # Получаем полные данные из auth-service
64 |         user_info = await auth_client.get_user(current_user.sub)
65 |
66 |         if user_info is None:
67 |             logger.error(
68 |                 f"User {current_user.sub} exists in JWT but not in auth-service. "
69 |                 f"Possible data inconsistency."
70 |             )

```

## src/main\_backend/api/v1/health.py (continued)

```

71 |         raise HTTPException(
72 |             status_code=status.HTTP_404_NOT_FOUND,
73 |             detail="User not found in auth-service",
74 |         )
75 |
76 |         # Проверяем, что пользователь активен
77 |         if not user_info.is_active:
78 |             logger.warning(f"User {current_user.sub} is disabled")
79 |             raise HTTPException(
80 |                 status_code=status.HTTP_403_FORBIDDEN,
81 |                 detail="User account is disabled",
82 |             )
83 |
84 |         return {
85 |             "status": "ok",
86 |             "source": "jwt + auth-service",
87 |             "user": {
88 |                 "uid": user_info.uid,
89 |                 "username": user_info.username,
90 |                 "email": user_info.email,
91 |                 "phone_number": user_info.phone_number,
92 |                 "role": user_info.role,
93 |                 "is_active": user_info.is_active,
94 |                 "is_email_verified": user_info.is_email_verified,
95 |                 "two_factor_enabled": user_info.two_factor_enabled,
96 |                 "created_at": user_info.created_at,
97 |             },
98 |             "jwt": {
99 |                 "jti": str(current_user.jti),
100 |                 "exp": current_user.exp.isoformat(),
101 |                 "iat": current_user.iat.isoformat(),
102 |             },
103 |         }
104 |
105 |     except AuthClientError as e:
106 |         logger.error(f"AuthClient error: {e}")
107 |         raise HTTPException(
108 |             status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
109 |             detail=f"Failed to fetch user data: {str(e)}",
110 |         )
111 |     except Exception as e:
112 |         logger.error(f"Unexpected error in /me: {e}", exc_info=True)
113 |         raise HTTPException(
114 |             status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
115 |             detail="Internal server error",
116 |         )
117 |
118 |
119 | @router.get("/admin-only")
120 | async def admin_only(
121 |     current_user: Annotated[CurrentUser, Depends(require_admin)],
122 | ):
123 |     """
124 |     Тестовый endpoint только для админов.
125 |
126 |     - Проверяет роль ADMIN или OWNER
127 |     - Возвращает 403 для обычных пользователей
128 |     """
129 |     return {
130 |         "status": "ok",

```

## src/main\_backend/api/v1/health.py (continued)

```
131 |         "message": f"Hello, admin {current_user.sub}!",
132 |         "role": current_user.role,
133 |     }
```

## src/main\_backend/app/dependencies.py

```
1 | # src/main_backend/app/dependencies.py
2 | """
3 | DI container (как в auth-service).
4 | """
5 | from __future__ import annotations
6 |
7 | from typing import Annotated, AsyncIterator
8 |
9 | from fastapi import Depends
10 | from sqlalchemy.ext.asyncio import AsyncSession
11 |
12 | from ..core.protocols import UnitOfWorkProtocol
13 | from ..external.database.config import async_session_factory
14 | from ..external.database.unit_of_work import UnitOfWork
15 | from ..external.integrations.auth_client import AuthClient
16 | from ..external.integrations.forecasting_client import ForecastingClient
17 |
18 |
19 | async def get_session() -> AsyncIterator[AsyncSession]:
20 |     """Dependency: async DB session."""
21 |     async with async_session_factory() as session:
22 |         yield session
23 |
24 |
25 | async def get_uow() -> AsyncIterator[UnitOfWorkProtocol]:
26 |     """Dependency: Unit of Work."""
27 |     uow = UnitOfWork()
28 |     async with uow:
29 |         yield uow
30 |
31 |
32 | def get_auth_client() -> AuthClient:
33 |     """Dependency: auth-service client."""
34 |     return AuthClient()
35 |
36 |
37 | def get_forecasting_client() -> ForecastingClient:
38 |     """Dependency: forecasting-service client."""
39 |     return ForecastingClient()
40 |
41 |
42 | # Type aliases для удобного использования
43 | UoWDep = Annotated[UnitOfWorkProtocol, Depends(get_uow)]
44 | AuthClientDep = Annotated[AuthClient, Depends(get_auth_client)]
45 | ForecastingClientDep = Annotated[ForecastingClient, Depends(get_forecasting_client)]
```

## src/main\_backend/app/middleware/exception\_handler.py

```

1 | # src/main_backend/app/middleware/exception_handler.py
2 | """
3 | Global exception handlers (как в auth-service).
4 | """
5 | import logging
6 | from typing import Any
7 |
8 | from fastapi import FastAPI, Request, status
9 | from fastapi.responses import JSONResponse
10 |
11 | from ...api.responses import ErrorDetail, ErrorResponse
12 | from ...core.exceptions import (
13 |     AuthServiceUnavailable,
14 |     ForecastingServiceUnavailable,
15 |     InvalidFeaturesError,
16 |     MainBackendError,
17 |     PredictionNotFoundError,
18 |     UserDisabledError,
19 |     UserNotFoundError,
20 | )
21 |
22 | logger = logging.getLogger(__name__)
23 |
24 |
25 | def _make_response(code: str, message: str, http_status: int) -> JSONResponse:
26 |     """Создать стандартизированный error response."""
27 |     return JSONResponse(
28 |         status_code=http_status,
29 |         content=ErrorResponse(
30 |             error=ErrorDetail(code=code, message=message)
31 |         ).model_dump(mode="json"),
32 |     )
33 |
34 |
35 | def register_exception_handlers(app: FastAPI) -> None:
36 |     """Регистрация обработчиков исключений."""
37 |
38 |     @app.exception_handler(PredictionNotFoundError)
39 |     async def prediction_not_found(request: Request, exc: PredictionNotFoundError):
40 |         return _make_response(exc.code, exc.message, status.HTTP_404_NOT_FOUND)
41 |
42 |     @app.exception_handler(UserNotFoundError)
43 |     async def user_not_found(request: Request, exc: UserNotFoundError):
44 |         return _make_response(exc.code, exc.message, status.HTTP_404_NOT_FOUND)
45 |
46 |     @app.exception_handler(UserDisabledError)
47 |     async def user_disabled(request: Request, exc: UserDisabledError):
48 |         return _make_response(exc.code, exc.message, status.HTTP_403_FORBIDDEN)
49 |
50 |     @app.exception_handler(InvalidFeaturesError)
51 |     async def invalid_features(request: Request, exc: InvalidFeaturesError):
52 |         return _make_response(exc.code, exc.message, status.HTTP_400_BAD_REQUEST)
53 |
54 |     @app.exception_handler(ForecastingServiceUnavailable)
55 |     async def forecasting_unavailable(
56 |         request: Request, exc: ForecastingServiceUnavailable
57 |     ):
58 |         logger.error(f"Forecasting service unavailable: {exc.message}")
59 |         return _make_response(

```

## src/main\_backend/app/middleware/exception\_handler.py (continued)

```

60 |         exc.code, exc.message, status.HTTP_503_SERVICE_UNAVAILABLE
61 |     )
62 |
63 |     @app.exception_handler(AuthServiceUnavailable)
64 |     async def auth_unavailable(request: Request, exc: AuthServiceUnavailable):
65 |         logger.error(f"Auth service unavailable: {exc.message}")
66 |         return _make_response(
67 |             exc.code, exc.message, status.HTTP_503_SERVICE_UNAVAILABLE
68 |         )
69 |
70 |     @app.exception_handler(MainBackendError)
71 |     async def main_backend_error(request: Request, exc: MainBackendError):
72 |         logger.error(f"Main backend error: {exc.message}", exc_info=True)
73 |         return _make_response(
74 |             exc.code, exc.message, status.HTTP_500_INTERNAL_SERVER_ERROR
75 |         )
76 |
77 |     @app.exception_handler(Exception)
78 |     async def unhandled_exception(request: Request, exc: Exception):
79 |         logger.exception(f"Unhandled exception: {exc}")
80 |         return _make_response(
81 |             "INTERNAL_ERROR",
82 |             "Unexpected error occurred",
83 |             status.HTTP_500_INTERNAL_SERVER_ERROR,
84 |         )

```

## src/main\_backend/app/middleware/request\_id.py

```

1 | # src/main_backend/app/middleware/request_id.py
2 | """
3 | Request ID middleware (как в auth-service).
4 | """
5 | import logging
6 | import uuid
7 |
8 | from fastapi import Request
9 | from starlette.middleware.base import BaseHTTPMiddleware
10 | from starlette.responses import Response
11 |
12 | logger = logging.getLogger(__name__)
13 |
14 |
15 | class RequestIDMiddleware(BaseHTTPMiddleware):
16 |     """Добавляет X-Request-ID к каждому запросу."""
17 |
18 |     async def dispatch(self, request: Request, call_next) -> Response:
19 |         request_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
20 |         request.state.request_id = request_id
21 |
22 |         # Добавляем request_id в лог-контекст
23 |         logger.info(
24 |             f"{request.method} {request.url.path}",
25 |             extra={"request_id": request_id},
26 |         )
27 |
28 |         response = await call_next(request)
29 |         response.headers["X-Request-ID"] = request_id
30 |         return response

```



## src/main\_backend/app/predictions/router.py

```

1 | # src/main_backend/app/predictions/router.py
2 | """
3 | Predictions API router.
4 | """
5 | from __future__ import annotations
6 |
7 | import logging
8 | from typing import Annotated
9 | from uuid import UUID
10 |
11 | from fastapi import APIRouter, Depends, Query, Request
12 |
13 | from ...api.responses import ApiResponse
14 | from ...core.security import CurrentUser, verify_access_token
15 | from ..dependencies import (
16 |     AuthClientDep,
17 |     ForecastingClientDep,
18 |     UowDep,
19 | )
20 | from .schemas import (
21 |     PredictionHistoryResponseSchema,
22 |     PredictionListItemSchema,
23 |     PredictionRequestSchema,
24 |     PredictionResponseSchema,
25 | )
26 | from .service import PredictionService
27 |
28 | logger = logging.getLogger(__name__)
29 |
30 | router = APIRouter(prefix="/api/v1/predictions", tags=["Predictions"])
31 |
32 |
33 | def get_prediction_service(
34 |     uow: UowDep,
35 |     auth_client: AuthClientDep,
36 |     forecasting_client: ForecastingClientDep,
37 | ) -> PredictionService:
38 |     """Dependency: PredictionService."""
39 |     return PredictionService(uow, auth_client, forecasting_client)
40 |
41 |
42 | ServiceDep = Annotated[PredictionService, Depends(get_prediction_service)]
43 |
44 |
45 | @router.post("/", response_model=ApiResponse[PredictionResponseSchema])
46 | async def create_prediction(
47 |     request: Request,
48 |     body: PredictionRequestSchema,
49 |     current_user: Annotated[CurrentUser, Depends(verify_access_token)],
50 |     service: ServiceDep,
51 | ):
52 |     """
53 |     Создать прогноз стоимости квартиры.
54 |
55 |     - **features**: характеристики квартиры
56 |     - **horizon**: now | 6_months | 1_year
57 |     """
58 |     entity = await service.create_prediction(
59 |         user_id=current_user.sub,

```

## src/main\_backend/app/predictions/router.py (continued)

```

60 |         features=body.features.model_dump(),
61 |         horizon=body.horizon,
62 |         ip_address=request.client.host if request.client else None,
63 |         user_agent=request.headers.get("user-agent"),
64 |     )
65 |     return ApiResponse(
66 |         data=PredictionResponseSchema.model_validate(entity)
67 |     )
68 |
69 |
70 | @router.get("/", response_model=ApiResponse[PredictionHistoryResponseSchema])
71 | async def get_history(
72 |     current_user: Annotated[CurrentUser, Depends(verify_access_token)],
73 |     service: ServiceDep,
74 |     limit: int = Query(default=20, ge=1, le=100),
75 |     offset: int = Query(default=0, ge=0),
76 | ):
77 |     """Получить историю прогнозов текущего пользователя."""
78 |     entities, total = await service.get_user_predictions(
79 |         user_id=current_user.sub,
80 |         limit=limit,
81 |         offset=offset,
82 |     )
83 |     items = [PredictionListItemSchema.from_entity(e) for e in entities]
84 |     return ApiResponse(
85 |         data=PredictionHistoryResponseSchema(
86 |             items=items,
87 |             total=total,
88 |             limit=limit,
89 |             offset=offset,
90 |         )
91 |     )
92 |
93 |
94 | @router.get("/{prediction_id}", response_model=ApiResponse[PredictionResponseSchema])
95 | async def get_prediction(
96 |     prediction_id: UUID,
97 |     current_user: Annotated[CurrentUser, Depends(verify_access_token)],
98 |     service: ServiceDep,
99 | ):
100 |     """Получить конкретный прогноз по ID."""
101 |     entity = await service.get_prediction(prediction_id, current_user.sub)
102 |     return ApiResponse(
103 |         data=PredictionResponseSchema.model_validate(entity)
104 |     )
105 |
106 |
107 | @router.delete("/{prediction_id}", response_model=ApiResponse[dict])
108 | async def delete_prediction(
109 |     prediction_id: UUID,
110 |     current_user: Annotated[CurrentUser, Depends(verify_access_token)],
111 |     service: ServiceDep,
112 | ):
113 |     """Удалить свой прогноз."""
114 |     await service.delete_prediction(prediction_id, current_user.sub)
115 |     return ApiResponse(data={"deleted": str(prediction_id)})

```

## src/main\_backend/app/predictions/schemas.py

```

1 | # src/main_backend/app/predictions/schemas.py
2 | """
3 | Pydantic schemas для predictions API.
4 | """
5 | from datetime import datetime
6 | from typing import Any, Optional
7 | from uuid import UUID
8 |
9 | from pydantic import BaseModel, ConfigDict, Field
10 |
11 | from ...core.constants import ForecastHorizon
12 |
13 |
14 | class FlatFeaturesSchema(BaseModel):
15 |     """Входные признаки квартиры."""
16 |
17 |     model_config = ConfigDict(extra="allow") # разрешаем one-hot признаки
18 |
19 |     # Числовые признаки
20 |     total_meters: float = Field(..., gt=0, le=1000)
21 |     living_meters: float = Field(..., ge=0)
22 |     kitchen_meters: float = Field(..., ge=0)
23 |     rooms_count: int = Field(..., ge=0, le=10)
24 |     floor: int = Field(..., ge=1)
25 |     floors_count: int = Field(..., ge=1)
26 |     floor_ratio: float = Field(..., ge=0, le=1)
27 |     living_ratio: float = Field(..., ge=0, le=1)
28 |     kitchen_ratio: float = Field(..., ge=0, le=1)
29 |     building_age: float = Field(..., ge=0)
30 |     year_of_construction: float = Field(..., ge=1850, le=2100)
31 |     latitude: float = Field(..., ge=-90, le=90)
32 |     longitude: float = Field(..., ge=-180, le=180)
33 |     dist_to_center_km: float = Field(..., ge=0)
34 |     dist_to_sea_km: float = Field(..., ge=0)
35 |     infrastructure_count: int = Field(..., ge=0)
36 |     security_score: int = Field(..., ge=0, le=5)
37 |
38 |     # Boolean признаки
39 |     has_intercom: int = Field(..., ge=0, le=1)
40 |     has_closed_territory: int = Field(..., ge=0, le=1)
41 |     has_code_door: int = Field(..., ge=0, le=1)
42 |     has_garage: int = Field(..., ge=0, le=1)
43 |     has_concierge: int = Field(..., ge=0, le=1)
44 |     offer_photos_count: int = Field(..., ge=0)
45 |     house_photos_count: int = Field(..., ge=0)
46 |     has_plan_photo: int = Field(..., ge=0, le=1)
47 |
48 |
49 | class PredictionRequestSchema(BaseModel):
50 |     """Запрос на прогнозирование."""
51 |
52 |     features: FlatFeaturesSchema
53 |     horizon: ForecastHorizon = ForecastHorizon.NOW
54 |
55 |
56 | class PredictionResponseSchema(BaseModel):
57 |     """Ответ с прогнозом."""
58 |
59 |     model_config = ConfigDict(from_attributes=True)

```

## src/main\_backend/app/predictions/schemas.py (continued)

```

60 |
61 |     id: UUID
62 |     user_id: UUID
63 |     predicted_price: float
64 |     predicted_price_per_sqm: float
65 |     horizon: ForecastHorizon
66 |     confidence: float
67 |     model_version: str
68 |     comparables: Optional[list[dict[str, Any]]] = None
69 |     created_at: datetime
70 |
71 |     @property
72 |     def trend_description(self) -> str:
73 |         """Описание тренда для отображения."""
74 |         return {
75 |             ForecastHorizon.NOW: "Текущая рыночная оценка",
76 |             ForecastHorizon.SIX_MONTHS: "Умеренный рост (+3.2%)",
77 |             ForecastHorizon.ONE_YEAR: "Прогноз на год (+6.8%)",
78 |         }[self.horizon]
79 |
80 |
81 | class PredictionListItemSchema(BaseModel):
82 |     """Элемент списка прогнозов."""
83 |
84 |     model_config = ConfigDict(from_attributes=True)
85 |
86 |     id: UUID
87 |     predicted_price: float
88 |     predicted_price_per_sqm: float
89 |     horizon: ForecastHorizon
90 |     total_meters: float
91 |     rooms_count: int
92 |     created_at: datetime
93 |
94 |     @classmethod
95 |     def from_entity(cls, entity) -> "PredictionListItemSchema":
96 |         features = entity.features
97 |         return cls(
98 |             id=entity.id,
99 |             predicted_price=entity.predicted_price,
100 |             predicted_price_per_sqm=entity.predicted_price_per_sqm,
101 |             horizon=entity.horizon,
102 |             total_meters=features.get("total_meters", 0),
103 |             rooms_count=features.get("rooms_count", 0),
104 |             created_at=entity.created_at,
105 |         )
106 |
107 |
108 | class PredictionHistoryResponseSchema(BaseModel):
109 |     """Ответ с историей прогнозов."""
110 |
111 |     items: list[PredictionListItemSchema]
112 |     total: int
113 |     limit: int
114 |     offset: int

```

## src/main\_backend/app/predictions/service.py

```

1 | # src/main_backend/app/predictions/service.py
2 | """
3 | PredictionService – orchestration layer.
4 | """
5 | from __future__ import annotations
6 |
7 | import logging
8 | from typing import Sequence
9 | from uuid import UUID
10 |
11 | from ...core.constants import MARKET_TRENDS, ForecastHorizon, PredictionStatus
12 | from ...core.exceptions import (
13 |     PredictionNotFoundError,
14 |     UserDisabledError,
15 |     UserNotFoundError,
16 | )
17 | from ...core.protocols import UnitOfWorkProtocol
18 | from ...domain.entities import PredictionEntity
19 | from ...external.integrations.auth_client import AuthClient
20 | from ...external.integrations.forecasting_client import ForecastingClient
21 |
22 | logger = logging.getLogger(__name__)
23 |
24 |
25 | class PredictionService:
26 |     """Сервис прогнозирования (оркестрация)."""
27 |
28 |     def __init__(
29 |         self,
30 |         uow: UnitOfWorkProtocol,
31 |         auth_client: AuthClient,
32 |         forecasting_client: ForecastingClient,
33 |     ):
34 |         self._uow = uow
35 |         self._auth_client = auth_client
36 |         self._forecasting_client = forecasting_client
37 |
38 |         # =====
39 |         # Public API
40 |         # =====
41 |
42 |     async def create_prediction(
43 |         self,
44 |         user_id: UUID,
45 |         features: dict,
46 |         horizon: ForecastHorizon,
47 |         ip_address: str | None = None,
48 |         user_agent: str | None = None,
49 |     ) -> PredictionEntity:
50 |         """
51 |         Создать прогноз.
52 |
53 |         Flow:
54 |         1. Проверить пользователя в auth-service
55 |         2. Получить базовую цену из forecasting-service (horizon=now)
56 |         3. Применить рыночный тренд
57 |         4. Сохранить в БД
58 |         """
59 |         # 1. Проверяем пользователя

```

## src/main\_backend/app/predictions/service.py (continued)

```

60 |         await self._verify_user(user_id)
61 |
62 |         # 2. Получаем базовую цену (всегда horizon=now)
63 |         try:
64 |             base_result = await self._forecasting_client.predict(
65 |                 features=features,
66 |                 horizon="now",
67 |             )
68 |         except Exception as e:
69 |             logger.error(f"Forecasting failed: {e}", exc_info=True)
70 |             # Сохраняем failed prediction для аудита
71 |             failed_entity = PredictionEntity.create_failed(
72 |                 user_id=user_id,
73 |                 features=features,
74 |                 horizon=horizon,
75 |                 error_message=str(e),
76 |                 ip_address=ip_address,
77 |                 user_agent=user_agent,
78 |             )
79 |             return await self._uow.predictions.create(failed_entity)
80 |
81 |         # 3. Применяем рыночный тренд
82 |         trend = MARKET_TRENDS[horizon]
83 |         predicted_price = base_result.predicted_price * trend["multiplier"]
84 |         predicted_price_per_sqm = (
85 |             base_result.predicted_price_per_sqm * trend["multiplier"]
86 |         )
87 |         confidence = max(0.0, base_result.confidence + trend["confidence_delta"])
88 |
89 |         # 4. Создаём и сохраняем entity
90 |         entity = PredictionEntity.create(
91 |             user_id=user_id,
92 |             features=features,
93 |             predicted_price=predicted_price,
94 |             predicted_price_per_sqm=predicted_price_per_sqm,
95 |             horizon=horizon,
96 |             confidence=confidence,
97 |             model_version=base_result.model_version,
98 |             ip_address=ip_address,
99 |             user_agent=user_agent,
100 |         )
101 |
102 |         return await self._uow.predictions.create(entity)
103 |
104 |     async def get_prediction(
105 |         self,
106 |         prediction_id: UUID,
107 |         user_id: UUID,
108 |     ) -> PredictionEntity:
109 |         """Получить прогноз (только свой)."""
110 |         entity = await self._uow.predictions.get_by_id(prediction_id)
111 |         if entity is None or entity.user_id != user_id:
112 |             raise PredictionNotFoundError(str(prediction_id))
113 |         return entity
114 |
115 |     async def get_user_predictions(
116 |         self,
117 |         user_id: UUID,
118 |         limit: int = 20,
119 |         offset: int = 0,

```

## src/main\_backend/app/predictions/service.py (continued)

```

120 |     ) -> tuple[Sequence[PredictionEntity], int]:
121 |         """Получить историю прогнозов пользователя."""
122 |         predictions = await self._uow.predictions.get_by_user(
123 |             user_id=user_id, limit=limit, offset=offset
124 |         )
125 |         total = await self._uow.predictions.count_by_user(user_id)
126 |         return predictions, total
127 |
128 |     async def delete_prediction(
129 |         self,
130 |         prediction_id: UUID,
131 |         user_id: UUID,
132 |     ) -> None:
133 |         """Удалить свой прогноз."""
134 |         deleted = await self._uow.predictions.delete(prediction_id, user_id)
135 |         if not deleted:
136 |             raise PredictionNotFoundError(str(prediction_id))
137 |
138 |     # =====
139 |     # Private helpers
140 |     # =====
141 |
142 |     async def _verify_user(self, user_id: UUID) -> None:
143 |         """Проверить, что пользователь существует и активен."""
144 |         user_info = await self._auth_client.get_user(user_id)
145 |         if user_info is None:
146 |             raise UserNotFoundError(str(user_id))
147 |         if not user_info.is_active:
148 |             raise UserDisabledError()

```

## src/main\_backend/config.py

```

1 | from pathlib import Path
2 | from functools import lru_cache
3 |
4 | from pydantic import Field, SecretStr, PostgresDsn, RedisDsn
5 | from pydantic_settings import BaseSettings, SettingsConfigDict
6 | from enum import Enum
7 |
8 | class Environment(str, Enum):
9 |     DEVELOPMENT = "development"
10 |     STAGING = "staging"
11 |     PRODUCTION = "production"
12 |     TESTING = "testing"
13 |
14 | class Settings(BaseSettings):
15 |
16 |     model_config = SettingsConfigDict(
17 |         env_file=".env",
18 |         env_file_encoding="utf-8",
19 |         case_sensitive=True,
20 |         extra="ignore",
21 |     )
22 |
23 |     APP_NAME: str = "real-estate-app"
24 |     APP_VERSION: str = "0.1.0"
25 |     ENVIRONMENT: Environment = Environment.DEVELOPMENT
26 |     DEBUG: bool = False
27 |

```

## src/main\_backend/config.py (continued)

```

28 |     POSTGRES_USER: str
29 |     POSTGRES_PASSWORD: SecretStr
30 |     POSTGRES_HOST: str
31 |     POSTGRES_PORT: str
32 |     POSTGRES_DB: str
33 |
34 |     @property
35 |     def db_url(self) -> str:
36 |         password = self.POSTGRES_PASSWORD.get_secret_value()
37 |         return (
38 |             f"postgresql+asyncpg://{self.POSTGRES_USER}:{password}"
39 |             f"@{self.POSTGRES_HOST}:{self.POSTGRES_PORT}"
40 |             f"/{self.POSTGRES_DB}"
41 |         )
42 |
43 |     JWT_ALGORITHM: str = "HS256"
44 |     JWT_ACCESS_SECRET_KEY: SecretStr
45 |     JWT_ISSUER: str = "auth-service"
46 |     JWT_AUDIENCE: str = "real-estate-app"
47 |
48 |     INTERNAL_SERVICE_TOKEN: str
49 |
50 |     AUTH_SERVICE_URL: str = "http://localhost:8001"
51 |     FORECASTING_SERVICE_URL: str = "http://localhost:8002"
52 |
53 |     AUTH_CLIENT_TIMEOUT: float = 5.0
54 |     FORECASTING_CLIENT_TIMEOUT: float = 30.0
55 |
56 |     REDIS_URL: RedisDsn = RedisDsn("redis://localhost:6379/0")
57 |     USER_CACHE_TTL: int = 300
58 |
59 |     CORS_ORIGINS: list[str] = [
60 |         "http://localhost:5173",
61 |         "http://localhost:3000",
62 |     ]
63 |     CORS_ALLOW_CREDENTIALS: bool = True
64 |
65 |     PREDICTION_RATE_LIMIT: int = 20 # максимум прогнозов
66 |     PREDICTION_RATE_WINDOW: int = 3600 # в час
67 |
68 |
69 | @lru_cache
70 | def get_settings() -> Settings:
71 |     """Singleton для настроек."""
72 |     return Settings()
73 |
74 |
75 | settings = get_settings()

```

## src/main\_backend/core/constants.py

```

1 | # src/main_backend/core/constants.py
2 | """
3 | Константы и перечисления домена.
4 | """
5 | from enum import Enum
6 |
7 |
8 | class UserRole(str, Enum):

```



## src/main\_backend/core/constants.py (continued)

```

9 |     """Роли пользователей (должны совпадать с auth-service)."""
10 |     USER = "USER"
11 |     MODERATOR = "MODERATOR"
12 |     ADMIN = "ADMIN"
13 |     OWNER = "OWNER"
14 |
15 |
16 | class ForecastHorizon(str, Enum):
17 |     """Горизонты прогнозирования."""
18 |     NOW = "now"
19 |     SIX_MONTHS = "6_months"
20 |     ONE_YEAR = "1_year"
21 |
22 |
23 | class PredictionStatus(str, Enum):
24 |     """Статусы прогнозов."""
25 |     PENDING = "pending"
26 |     SUCCESS = "success"
27 |     FAILED = "failed"
28 |
29 |
30 | # Рыночные тренды (упрощённо, в идеале — из внешнего источника)
31 | MARKET_TRENDS = {
32 |     ForecastHorizon.NOW: {"multiplier": 1.0, "confidence_delta": 0.0},
33 |     ForecastHorizon.SIX_MONTHS: {"multiplier": 1.032, "confidence_delta": -0.05},
34 |     ForecastHorizon.ONE_YEAR: {"multiplier": 1.068, "confidence_delta": -0.13},
35 | }

```

## src/main\_backend/core/exceptions.py

```

1 | # src/main_backend/core/exceptions.py
2 | """
3 | Domain exceptions (как в auth-service).
4 | """
5 |
6 |
7 | class MainBackendError(Exception):
8 |     """Базовая ошибка сервиса."""
9 |     def __init__(self, message: str = "Internal error", code: str = "INTERNAL_ERROR"):
10 |         self.message = message
11 |         self.code = code
12 |         super().__init__(message)
13 |
14 |
15 | class PredictionError(MainBackendError):
16 |     """Ошибка прогнозирования."""
17 |     def __init__(self, message: str = "Prediction failed"):
18 |         super().__init__(message=message, code="PREDICTION_ERROR")
19 |
20 |
21 | class PredictionNotFoundError(MainBackendError):
22 |     """Прогноз не найден."""
23 |     def __init__(self, prediction_id: str):
24 |         super().__init__(
25 |             message=f"Prediction {prediction_id} not found",
26 |             code="PREDICTION_NOT_FOUND",
27 |         )
28 |
29 |

```

## src/main\_backend/core/exceptions.py (continued)

```

30 | class ForecastingServiceUnavailable(MainBackendError):
31 |     """Forecasting service недоступен."""
32 |     def __init__(self, details: str = ""):
33 |         super().__init__(
34 |             message=f"Forecasting service unavailable: {details}",
35 |             code="FORECASTING_UNAVAILABLE",
36 |         )
37 |
38 |
39 | class AuthServiceUnavailable(MainBackendError):
40 |     """Auth service недоступен."""
41 |     def __init__(self, details: str = ""):
42 |         super().__init__(
43 |             message=f"Auth service unavailable: {details}",
44 |             code="AUTH_SERVICE_UNAVAILABLE",
45 |         )
46 |
47 |
48 | class UserNotFoundError(MainBackendError):
49 |     """Пользователь не найден в auth-service."""
50 |     def __init__(self, user_id: str):
51 |         super().__init__(
52 |             message=f"User {user_id} not found in auth-service",
53 |             code="USER_NOT_FOUND",
54 |         )
55 |
56 |
57 | class UserDisabledError(MainBackendError):
58 |     """Пользователь деактивирован."""
59 |     def __init__(self):
60 |         super().__init__(
61 |             message="User account is disabled",
62 |             code="USER_DISABLED",
63 |         )
64 |
65 |
66 | class InvalidFeaturesError(MainBackendError):
67 |     """Невалидные входные признаки."""
68 |     def __init__(self, details: str = ""):
69 |         super().__init__(
70 |             message=f"Invalid flat features: {details}",
71 |             code="INVALID_FEATURES",
72 |         )

```

## src/main\_backend/core/protocols.py

```

1 | # src/main_backend/core/protocols.py
2 | """
3 | Интерфейсы (Protocol) для DI. Позволяют подменять реализации в тестах.
4 | """
5 | from __future__ import annotations
6 |
7 | from types import TracebackType
8 | from typing import Optional, Protocol, Self, Sequence
9 | from uuid import UUID
10 |
11 |
12 | class PredictionRepositoryProtocol(Protocol):
13 |     """Интерфейс репозитория прогнозов."""

```

## src/main\_backend/core/protocols.py (continued)

```

14 |
15 |     async def create(self, entity: "PredictionEntity") -> "PredictionEntity": ...
16 |     async def get_by_id(self, prediction_id: UUID) -> Optional["PredictionEntity"]: ...
17 |     async def get_by_user(
18 |         self,
19 |         user_id: UUID,
20 |         limit: int = 20,
21 |         offset: int = 0,
22 |     ) -> Sequence["PredictionEntity"]: ...
23 |     async def delete(self, prediction_id: UUID, user_id: UUID) -> bool: ...
24 |     async def count_by_user(self, user_id: UUID) -> int: ...
25 |
26 |
27 | class UnitOfWorkProtocol(Protocol):
28 |     """Unit of Work паттерн."""
29 |
30 |     predictions: PredictionRepositoryProtocol
31 |
32 |     async def __aenter__(self) -> Self: ...
33 |     async def __aexit__(
34 |         self,
35 |         exc_type: Optional[type[BaseException]],
36 |         exc_val: Optional[BaseException],
37 |         exc_tb: Optional[TracebackType],
38 |     ) -> None: ...
39 |     async def commit(self) -> None: ...
40 |     async def rollback(self) -> None: ...
41 |
42 |
43 | class AuthClientProtocol(Protocol):
44 |     """Интерфейс клиента auth-service."""
45 |
46 |     async def get_user(self, user_id: UUID) -> Optional["UserInfo"]: ...
47 |     async def invalidate_cache(self, user_id: UUID) -> None: ...
48 |
49 |
50 | class ForecastingClientProtocol(Protocol):
51 |     """Интерфейс клиента forecasting-service."""
52 |
53 |     async def predict(self, features: dict, horizon: str) -> "PredictionResult": ...

```

## src/main\_backend/core/security.py

```

1 | from __future__ import annotations
2 |
3 | import logging
4 | from dataclasses import dataclass
5 | from datetime import datetime, timezone
6 | from typing import Annotated, Callable
7 | from uuid import UUID
8 |
9 | from fastapi import Depends, HTTPException, status
10 | from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
11 | from jose import JWTError, jwt
12 |
13 | from main_backend.config import settings
14 |
15 | logger = logging.getLogger(__name__)
16 |

```

## src/main\_backend/core/security.py (continued)

```

17 | bearer_scheme = HTTPBearer(auto_error=True)
18 |
19 |
20 | @dataclass(frozen=True)
21 | class CurrentUser:
22 |     """Данные текущего пользователя, извлечённые из JWT."""
23 |     sub: UUID          # user UID
24 |     role: str          # USER, MODERATOR, ADMIN, OWNER
25 |     jti: UUID          # уникальный ID токена (для blacklist)
26 |     exp: datetime      # expiration
27 |     iat: datetime      # issued at
28 |     token_type: str    # всегда "access"
29 |
30 |
31 | async def verify_access_token(
32 |     credentials: Annotated[HTTPAuthorizationCredentials, Depends(bearer_scheme)],
33 | ) -> CurrentUser:
34 |     """
35 |     Валидация JWT access токена.
36 |
37 |     Проверяет:
38 |     - Подпись (HS256)
39 |     - Срок действия (exp)
40 |     - Issuer (iss)
41 |     - Audience (aud)
42 |     - Тип токена (type == "access")
43 |     - Обязательные поля (sub, role, jti)
44 |
45 |     Raises:
46 |         HTTPException 401 - при любой ошибке валидации
47 |     """
48 |     token = credentials.credentials
49 |
50 |     try:
51 |         # 1. Декодируем и валидируем JWT
52 |         payload = jwt.decode(
53 |             token,
54 |             settings.JWT_ACCESS_SECRET_KEY.get_secret_value(),
55 |             algorithms=[settings.JWT_ALGORITHM],
56 |             issuer=settings.JWT_ISSUER,
57 |             audience=settings.JWT_AUDIENCE,
58 |             options={
59 |                 "verify_exp": True,
60 |                 "verify_iss": True,
61 |                 "verify_aud": True,
62 |                 "require": ["sub", "exp", "iat", "iss", "aud", "jti", "type", "role"],
63 |             },
64 |         )
65 |
66 |         # 2. Проверяем тип токена (должен быть "access", а не "auth")
67 |         token_type = payload.get("type")
68 |         if token_type != "access":
69 |             logger.warning(f"Invalid token type: {token_type}")
70 |             raise HTTPException(
71 |                 status_code=status.HTTP_401_UNAUTHORIZED,
72 |                 detail="Invalid token type. Expected access token.",
73 |                 headers={"WWW-Authenticate": "Bearer"},
74 |             )
75 |
76 |         # 3. Извлекаем и валидируем поля

```

## src/main\_backend/core/security.py (continued)

```

77 |         try:
78 |             sub = UUID(payload["sub"])
79 |             jti = UUID(payload["jti"])
80 |         except (ValueError, KeyError) as e:
81 |             logger.warning(f"Invalid UUID in token: {e}")
82 |             raise HTTPException(
83 |                 status_code=status.HTTP_401_UNAUTHORIZED,
84 |                 detail="Invalid token payload",
85 |                 headers={"WWW-Authenticate": "Bearer"},
86 |             )
87 |
88 |         role = payload.get("role")
89 |         if not role or not isinstance(role, str):
90 |             raise HTTPException(
91 |                 status_code=status.HTTP_401_UNAUTHORIZED,
92 |                 detail="Invalid role in token",
93 |                 headers={"WWW-Authenticate": "Bearer"},
94 |             )
95 |
96 |         # 4. Конвертируем timestamps
97 |         exp = datetime.fromtimestamp(payload["exp"], tz=timezone.utc)
98 |         iat = datetime.fromtimestamp(payload["iat"], tz=timezone.utc)
99 |
100 |         return CurrentUser(
101 |             sub=sub,
102 |             role=role,
103 |             jti=jti,
104 |             exp=exp,
105 |             iat=iat,
106 |             token_type=token_type,
107 |         )
108 |
109 |     except jwt.ExpiredSignatureError:
110 |         logger.info("Token expired")
111 |         raise HTTPException(
112 |             status_code=status.HTTP_401_UNAUTHORIZED,
113 |             detail="Token has expired. Please login again.",
114 |             headers={"WWW-Authenticate": "Bearer"},
115 |         )
116 |     except jwt.JWTClaimsError as e:
117 |         logger.warning(f"JWT claims error: {e}")
118 |         raise HTTPException(
119 |             status_code=status.HTTP_401_UNAUTHORIZED,
120 |             detail=f"Invalid token claims: {e}",
121 |             headers={"WWW-Authenticate": "Bearer"},
122 |         )
123 |     except JWTErrors as e:
124 |         logger.warning(f"JWT validation failed: {e}")
125 |         raise HTTPException(
126 |             status_code=status.HTTP_401_UNAUTHORIZED,
127 |             detail="Invalid or malformed token",
128 |             headers={"WWW-Authenticate": "Bearer"},
129 |         )
130 |
131 |
132 | # =====
133 | # Dependencies для проверки ролей
134 | # =====
135 |
136 | def require_roles(*allowed_roles: str) -> Callable:

```

## src/main\_backend/core/security.py (continued)

```

137 | """
138 |     Фабрика dependency для проверки ролей.
139 |
140 |     Пример использования:
141 |         @router.get("/admin", dependencies=[Depends(require_roles("ADMIN", "OWNER"))])
142 |         async def admin_only(): ...
143 | """
144 |
145 | async def _check_role(
146 |     current_user: Annotated[CurrentUser, Depends(verify_access_token)],
147 | ) -> CurrentUser:
148 |     if current_user.role not in allowed_roles:
149 |         logger.warning(
150 |             f"Access denied for user {current_user.sub}: "
151 |             f"role={current_user.role}, required={allowed_roles}"
152 |         )
153 |         raise HTTPException(
154 |             status_code=status.HTTP_403_FORBIDDEN,
155 |             detail=f"Access denied. Required role: {'', '.join(allowed_roles)}",
156 |         )
157 |     return current_user
158 |
159 |     return _check_role
160 |
161 | # Готовые dependency для типичных случаев
162 | require_user = require_roles("USER", "MODERATOR", "ADMIN", "OWNER")
163 | require_moderator = require_roles("MODERATOR", "ADMIN", "OWNER")
164 | require_admin = require_roles("ADMIN", "OWNER")
165 | require_owner = require_roles("OWNER")
166 |
167 |
168 | # =====
169 | # Утилита для получения только user_id (самая частая операция)
170 | # =====
171 |
172 | async def get_current_user_id(
173 |     current_user: Annotated[CurrentUser, Depends(verify_access_token)],
174 | ) -> UUID:
175 |     """Возвращает только UUID текущего пользователя."""
176 |     return current_user.sub

```

## src/main\_backend/dependencies.py

```

1 | # src/main_backend/dependencies.py
2 | """
3 | Dependency Injection контейнер для main-backend.
4 | """
5 | from __future__ import annotations
6 |
7 | from typing import Annotated
8 |
9 | from fastapi import Depends
10 |
11 | from .integrations.auth_client import AuthClient
12 |
13 |
14 | def get_auth_client() -> AuthClient:
15 |     """Dependency: получить AuthClient."""
16 |     return AuthClient()

```

## src/main\_backend/dependencies.py (continued)

```

17 |
18 |
19 | # Type alias для удобного использования в endpoints
20 | AuthClientDep = Annotated[AuthClient, Depends(get_auth_client)]

```

## src/main\_backend/domain/entities.py

```

1 | # src/main_backend/domain/entities.py
2 | """
3 | Domain entities (как UserEntity в auth-service).
4 | """
5 | from __future__ import annotations
6 |
7 | from dataclasses import dataclass, field
8 | from datetime import datetime
9 | from typing import Any, Optional
10 | from uuid import UUID, uuid4
11 |
12 | from ..core.constants import ForecastHorizon, PredictionStatus
13 |
14 |
15 | @dataclass
16 | class PredictionEntity:
17 |     """Сущность прогноза стоимости квартиры."""
18 |
19 |     id: UUID
20 |     user_id: UUID
21 |     features: dict[str, Any]
22 |     predicted_price: float
23 |     predicted_price_per_sqm: float
24 |     horizon: ForecastHorizon
25 |     confidence: float
26 |     model_version: str
27 |     status: PredictionStatus
28 |     comparables: Optional[list[dict]] = None
29 |     error_message: Optional[str] = None
30 |     ip_address: Optional[str] = None
31 |     user_agent: Optional[str] = None
32 |     created_at: datetime = field(default_factory=datetime.utcnow)
33 |     updated_at: datetime = field(default_factory=datetime.utcnow)
34 |
35 |     @classmethod
36 |     def create(
37 |         cls,
38 |         user_id: UUID,
39 |         features: dict[str, Any],
40 |         predicted_price: float,
41 |         predicted_price_per_sqm: float,
42 |         horizon: ForecastHorizon,
43 |         confidence: float,
44 |         model_version: str,
45 |         comparables: Optional[list[dict]] = None,
46 |         ip_address: Optional[str] = None,
47 |         user_agent: Optional[str] = None,
48 |     ) -> "PredictionEntity":
49 |         """Фабричный метод для создания нового прогноза."""
50 |         return cls(
51 |             id=uuid4(),
52 |             user_id=user_id,

```

## src/main\_backend/domain/entities.py (continued)

```

53 |         features=features,
54 |         predicted_price=predicted_price,
55 |         predicted_price_per_sqm=predicted_price_per_sqm,
56 |         horizon=horizon,
57 |         confidence=confidence,
58 |         model_version=model_version,
59 |         status=PredictionStatus.SUCCESS,
60 |         comparables=comparables,
61 |         ip_address=ip_address,
62 |         user_agent=user_agent,
63 |     )
64 |
65 |     @classmethod
66 |     def create_failed(
67 |         cls,
68 |         user_id: UUID,
69 |         features: dict[str, Any],
70 |         horizon: ForecastHorizon,
71 |         error_message: str,
72 |         ip_address: Optional[str] = None,
73 |         user_agent: Optional[str] = None,
74 |     ) -> "PredictionEntity":
75 |         """Создание записи о неудачном прогнозе (для аудита)."""
76 |         return cls(
77 |             id=uuid4(),
78 |             user_id=user_id,
79 |             features=features,
80 |             predicted_price=0.0,
81 |             predicted_price_per_sqm=0.0,
82 |             horizon=horizon,
83 |             confidence=0.0,
84 |             model_version="unknown",
85 |             status=PredictionStatus.FAILED,
86 |             error_message=error_message,
87 |             ip_address=ip_address,
88 |             user_agent=user_agent,
89 |         )

```

## src/main\_backend/external/database/config.py

```

1 | # src/main_backend/external/database/config.py
2 | """
3 | Database engine и session factory.
4 | """
5 | from sqlalchemy.ext.asyncio import (
6 |     AsyncSession,
7 |     async_sessionmaker,
8 |     create_async_engine,
9 | )
10 |
11 | from ...config import settings
12 |
13 | engine = create_async_engine(
14 |     settings.db_url,
15 |     echo=settings.DEBUG,
16 |     pool_size=10,
17 |     max_overflow=20,
18 |     pool_pre_ping=True,
19 | )

```



## src/main\_backend/external/database/config.py (continued)

```

20 |
21 | async_session_factory = async_sessionmaker(
22 |     engine,
23 |     class_=AsyncSession,
24 |     expire_on_commit=False,
25 | )
26 |
27 |
28 | async def get_session() -> AsyncSession:
29 |     """Dependency: асинхронная сессия."""
30 |     async with async_session_factory() as session:
31 |         yield session

```

## src/main\_backend/external/database/models.py

```

1 | # src/main_backend/external/database/models.py
2 | """
3 | SQLAlchemy ORM models.
4 | """
5 | from datetime import datetime
6 | from typing import Optional
7 | from uuid import UUID, uuid4
8 |
9 | from sqlalchemy import (
10 |     BigInteger,
11 |     DateTime,
12 |     Float,
13 |     Index,
14 |     String,
15 |     Text,
16 |     func,
17 | )
18 | from sqlalchemy.dialects.postgresql import INET, JSONB, UUID as PG_UUID
19 | from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
20 |
21 | from ...core.constants import ForecastHorizon, PredictionStatus
22 |
23 |
24 | class Base(DeclarativeBase):
25 |     """Базовый класс для всех моделей."""
26 |     pass
27 |
28 |
29 | class PredictionModel(Base):
30 |     """ORM модель прогноза."""
31 |
32 |     __tablename__ = "predictions"
33 |     __table_args__ = (
34 |         Index("idx_predictions_user_id", "user_id"),
35 |         Index("idx_predictions_created_at", "created_at"),
36 |         Index("idx_predictions_status", "status"),
37 |     )
38 |
39 |     id: Mapped[UUID] = mapped_column(
40 |         PG_UUID(as_uuid=True),
41 |         primary_key=True,
42 |         default=uuid4,
43 |     )
44 |     user_id: Mapped[UUID] = mapped_column(

```

## src/main\_backend/external/database/models.py (continued)

```

45 |         PG_UUID(as_uuid=True),
46 |         nullable=False,
47 |         index=True,
48 |     )
49 |     features: Mapped[dict] = mapped_column(JSONB, nullable=False)
50 |     predicted_price: Mapped[float] = mapped_column(Float, nullable=False)
51 |     predicted_price_per_sqm: Mapped[float] = mapped_column(Float, nullable=False)
52 |     horizon: Mapped[str] = mapped_column(
53 |         String(20),
54 |         nullable=False,
55 |         default=ForecastHorizon.NOW.value,
56 |     )
57 |     confidence: Mapped[float] = mapped_column(Float, nullable=False)
58 |     model_version: Mapped[str] = mapped_column(String(50), nullable=False)
59 |     status: Mapped[str] = mapped_column(
60 |         String(20),
61 |         nullable=False,
62 |         default=PredictionStatus.SUCCESS.value,
63 |     )
64 |     comparables: Mapped[Optional[list]] = mapped_column(JSONB, nullable=True)
65 |     error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
66 |     ip_address: Mapped[Optional[str]] = mapped_column(INET, nullable=True)
67 |     user_agent: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
68 |     created_at: Mapped[datetime] = mapped_column(
69 |         DateTime(timezone=True),
70 |         server_default=func.now(),
71 |         nullable=False,
72 |     )
73 |     updated_at: Mapped[datetime] = mapped_column(
74 |         DateTime(timezone=True),
75 |         server_default=func.now(),
76 |         onupdate=func.now(),
77 |         nullable=False,
78 |     )
79 |
80 |     def __repr__(self) -> str:
81 |         return (
82 |             f"<Prediction id={self.id} user={self.user_id} "
83 |             f"status={self.status} price={self.predicted_price}>"
84 |         )

```

## src/main\_backend/external/database/repositories.py

```

1 | # src/main_backend/external/database/repositories.py
2 | """
3 | Repository implementations (как в auth-service).
4 | """
5 | from __future__ import annotations
6 |
7 | import logging
8 | from typing import Optional, Sequence
9 | from uuid import UUID
10 |
11 | from sqlalchemy import delete, func, select
12 | from sqlalchemy.ext.asyncio import AsyncSession
13 |
14 | from ...core.exceptions import PredictionNotFoundError
15 | from ...domain.entities import PredictionEntity
16 | from .models import PredictionModel

```

## src/main\_backend/external/database/repositories.py (continued)

```

17 |
18 | logger = logging.getLogger(__name__)
19 |
20 |
21 | class PredictionRepository:
22 |     """Репозиторий для работы с прогнозами."""
23 |
24 |     def __init__(self, session: AsyncSession):
25 |         self._session = session
26 |
27 |     # =====
28 |     # Мэппинг Entity <-> ORM Model
29 |     # =====
30 |
31 |     @staticmethod
32 |     def _to_entity(model: PredictionModel) -> PredictionEntity:
33 |         """Конвертация ORM модели в domain entity."""
34 |         from ...core.constants import ForecastHorizon, PredictionStatus
35 |
36 |         return PredictionEntity(
37 |             id=model.id,
38 |             user_id=model.user_id,
39 |             features=model.features,
40 |             predicted_price=model.predicted_price,
41 |             predicted_price_per_sqm=model.predicted_price_per_sqm,
42 |             horizon=ForecastHorizon(model.horizon),
43 |             confidence=model.confidence,
44 |             model_version=model.model_version,
45 |             status=PredictionStatus(model.status),
46 |             comparables=model.comparables,
47 |             error_message=model.error_message,
48 |             ip_address=model.ip_address,
49 |             user_agent=model.user_agent,
50 |             created_at=model.created_at,
51 |             updated_at=model.updated_at,
52 |         )
53 |
54 |     @staticmethod
55 |     def _to_model(entity: PredictionEntity) -> PredictionModel:
56 |         """Конвертация domain entity в ORM модель."""
57 |         return PredictionModel(
58 |             id=entity.id,
59 |             user_id=entity.user_id,
60 |             features=entity.features,
61 |             predicted_price=entity.predicted_price,
62 |             predicted_price_per_sqm=entity.predicted_price_per_sqm,
63 |             horizon=entity.horizon.value,
64 |             confidence=entity.confidence,
65 |             model_version=entity.model_version,
66 |             status=entity.status.value,
67 |             comparables=entity.comparables,
68 |             error_message=entity.error_message,
69 |             ip_address=entity.ip_address,
70 |             user_agent=entity.user_agent,
71 |             created_at=entity.created_at,
72 |             updated_at=entity.updated_at,
73 |         )
74 |
75 |     # =====
76 |     # CRUD operations

```

## src/main\_backend/external/database/repositories.py (continued)

```

77 | # =====
78 |
79 | async def create(self, entity: PredictionEntity) -> PredictionEntity:
80 |     """Создать прогноз."""
81 |     model = self._to_model(entity)
82 |     self._session.add(model)
83 |     await self._session.flush()
84 |     await self._session.refresh(model)
85 |     logger.info(f"Created prediction {model.id} for user {model.user_id}")
86 |     return self._to_entity(model)
87 |
88 | async def get_by_id(self, prediction_id: UUID) -> Optional[PredictionEntity]:
89 |     """Получить прогноз по ID."""
90 |     stmt = select(PredictionModel).where(PredictionModel.id == prediction_id)
91 |     result = await self._session.execute(stmt)
92 |     model = result.scalar_one_or_none()
93 |     return self._to_entity(model) if model else None
94 |
95 | async def get_by_user(
96 |     self,
97 |     user_id: UUID,
98 |     limit: int = 20,
99 |     offset: int = 0,
100 | ) -> Sequence[PredictionEntity]:
101 |     """Получить прогнозы пользователя с пагинацией."""
102 |     stmt = (
103 |         select(PredictionModel)
104 |         .where(PredictionModel.user_id == user_id)
105 |         .order_by(PredictionModel.created_at.desc())
106 |         .limit(limit)
107 |         .offset(offset)
108 |     )
109 |     result = await self._session.execute(stmt)
110 |     models = result.scalars().all()
111 |     return [self._to_entity(m) for m in models]
112 |
113 | async def delete(self, prediction_id: UUID, user_id: UUID) -> bool:
114 |     """Удалить прогноз пользователя."""
115 |     stmt = delete(PredictionModel).where(
116 |         PredictionModel.id == prediction_id,
117 |         PredictionModel.user_id == user_id,
118 |     )
119 |     result = await self._session.execute(stmt)
120 |     deleted = result.rowcount > 0
121 |     if deleted:
122 |         logger.info(f"Deleted prediction {prediction_id} by user {user_id}")
123 |     return deleted
124 |
125 | async def count_by_user(self, user_id: UUID) -> int:
126 |     """Посчитать количество прогнозов пользователя."""
127 |     stmt = (
128 |         select(func.count())
129 |         .select_from(PredictionModel)
130 |         .where(PredictionModel.user_id == user_id)
131 |     )
132 |     result = await self._session.execute(stmt)
133 |     return result.scalar_one()

```

## src/main\_backend/external/database/unit\_of\_work.py

```

1 | # src/main_backend/external/database/unit_of_work.py
2 | """
3 | Unit of Work pattern (как в auth-service).
4 | """
5 | from __future__ import annotations
6 |
7 | import logging
8 | from types import TracebackType
9 | from typing import Optional, Self
10 |
11 | from sqlalchemy.ext.asyncio import AsyncSession
12 |
13 | from ...core.protocols import UnitOfWorkProtocol
14 | from .config import async_session_factory
15 | from .repositories import PredictionRepository
16 |
17 | logger = logging.getLogger(__name__)
18 |
19 |
20 | class UnitOfWork:
21 |     """Unit of Work для управления транзакциями."""
22 |
23 |     def __init__(self):
24 |         self._session: Optional[AsyncSession] = None
25 |         self.predictions: PredictionRepository
26 |
27 |     async def __aenter__(self) -> Self:
28 |         self._session = async_session_factory()
29 |         self.predictions = PredictionRepository(self._session)
30 |         return self
31 |
32 |     async def __aexit__(
33 |         self,
34 |         exc_type: Optional[type[BaseException]],
35 |         exc_val: Optional[BaseException],
36 |         exc_tb: Optional[TracebackType],
37 |     ) -> None:
38 |         if self._session is None:
39 |             return
40 |
41 |         if exc_type is not None:
42 |             await self.rollback()
43 |             logger.warning(f"Transaction rolled back due to: {exc_val}")
44 |         else:
45 |             await self.commit()
46 |
47 |         await self._session.close()
48 |
49 |     async def commit(self) -> None:
50 |         if self._session is None:
51 |             return
52 |         await self._session.commit()
53 |
54 |     async def rollback(self) -> None:
55 |         if self._session is None:
56 |             return
57 |         await self._session.rollback()

```

## src/main\_backend/external/integrations/auth\_client.py

```

1 | # src/main_backend/integrations/auth_client.py
2 | """
3 | HTTP клиент для взаимодействия с auth-service.
4 | Без кэширования – каждый запрос идёт напрямую в auth-service.
5 | """
6 | from __future__ import annotations
7 |
8 | import logging
9 | from typing import Optional
10 | from uuid import UUID
11 |
12 | import httpx
13 | from pydantic import BaseModel
14 | from tenacity import (
15 |     retry,
16 |     retry_if_exception_type,
17 |     stop_after_attempt,
18 |     wait_exponential,
19 | )
20 |
21 | from main_backend.config import settings
22 |
23 | logger = logging.getLogger(__name__)
24 |
25 |
26 | class UserInfo(BaseModel):
27 |     """Публичные данные пользователя из auth-service."""
28 |     uid: str
29 |     username: str
30 |     email: str
31 |     phone_number: Optional[str] = None
32 |     role: str
33 |     is_active: bool
34 |     is_email_verified: bool
35 |     two_factor_enabled: bool
36 |     created_at: str
37 |
38 |
39 | class AuthClientError(Exception):
40 |     """Базовая ошибка AuthClient."""
41 |     pass
42 |
43 |
44 | class UserNotFoundError(AuthClientError):
45 |     """Пользователь не найден в auth-service."""
46 |     pass
47 |
48 |
49 | class AuthClient:
50 |     """
51 |     HTTP клиент для auth-service.
52 |
53 |     Каждый запрос идёт напрямую в auth-service через internal API.
54 |     Для диплома этого достаточно – нагрузка низкая.
55 |     """
56 |
57 |     def __init__(self):
58 |         self._base_url = settings.AUTH_SERVICE_URL.rstrip("/")
59 |         self._internal_token = settings.INTERNAL_SERVICE_TOKEN

```

## src/main\_backend/external/integrations/auth\_client.py (continued)

```

60 |         self._timeout = settings.AUTH_CLIENT_TIMEOUT
61 |
62 |         # =====
63 |         # Public API
64 |         # =====
65 |
66 |         @retry(
67 |             stop=stop_after_attempt(3),
68 |             wait=wait_exponential(multiplier=1, min=1, max=5),
69 |             retry=retry_if_exception_type(
70 |                 (httpx.TimeoutException, httpx.ConnectError)
71 |             ),
72 |             reraise=True,
73 |         )
74 |         async def get_user(self, user_id: UUID) -> Optional[UserInfo]:
75 |             """
76 |             Получает информацию о пользователе из auth-service.
77 |
78 |             Returns:
79 |                 UserInfo если пользователь найден
80 |                 None если пользователь не существует
81 |             """
82 |             url = f"{self._base_url}/api/v1/internal/users/{user_id}"
83 |             headers = {
84 |                 "X-Internal-Token": self._internal_token,
85 |                 "Accept": "application/json",
86 |             }
87 |
88 |             logger.debug(f"Fetching user {user_id} from auth-service")
89 |
90 |             async with httpx.AsyncClient(timeout=self._timeout) as client:
91 |                 try:
92 |                     response = await client.get(url, headers=headers)
93 |
94 |                     if response.status_code == 404:
95 |                         logger.warning(f"User {user_id} not found in auth-service")
96 |                         return None
97 |
98 |                     if response.status_code == 401:
99 |                         logger.error(
100 |                             "Invalid INTERNAL_SERVICE_TOKEN. "
101 |                             "Check that it matches in auth-service and main-backend."
102 |                         )
103 |                         raise AuthClientError("Internal authentication failed")
104 |
105 |                     response.raise_for_status()
106 |
107 |                     body = response.json()
108 |                     user_data = body.get("data")
109 |
110 |                     if not user_data:
111 |                         logger.error(f"Empty user data for {user_id}")
112 |                         return None
113 |
114 |                     return UserInfo.model_validate(user_data)
115 |
116 |             except httpx.TimeoutException:
117 |                 logger.error(f"Timeout fetching user {user_id}")
118 |                 raise
119 |             except httpx.ConnectError:

```

## src/main\_backend/external/integrations/auth\_client.py (continued)

```

120 |         logger.error(
121 |             f"Cannot connect to auth-service at {self._base_url}. "
122 |             f"Is auth-service running?"
123 |         )
124 |         raise
125 |     except httpx.HTTPStatusError as e:
126 |         logger.error(
127 |             f"HTTP error fetching user {user_id}: "
128 |             f"{e.response.status_code} {e.response.text}"
129 |         )
130 |         raise AuthClientError(
131 |             f"Auth-service returned {e.response.status_code}"
132 |         )
133 |     except AuthClientError:
134 |         raise
135 |     except Exception as e:
136 |         logger.error(f"Unexpected error fetching user {user_id}: {e}")
137 |         raise AuthClientError(f"Unexpected error: {e}")

```

## src/main\_backend/external/integrations/forecasting\_client.py

```

1 | # src/main_backend/external/integrations/forecasting_client.py
2 | """
3 | HTTP client для forecasting-service.
4 | """
5 | from __future__ import annotations
6 |
7 | import logging
8 | from dataclasses import dataclass
9 |
10 | import httpx
11 | from tenacity import (
12 |     retry,
13 |     retry_if_exception_type,
14 |     stop_after_attempt,
15 |     wait_exponential,
16 | )
17 |
18 | from ...config import settings
19 | from ...core.exceptions import ForecastingServiceUnavailable
20 |
21 | logger = logging.getLogger(__name__)
22 |
23 |
24 | @dataclass
25 | class ForecastingResult:
26 |     """Результат от forecasting-service."""
27 |
28 |     predicted_price: float
29 |     predicted_price_per_sqm: float
30 |     confidence: float
31 |     model_version: str
32 |
33 |
34 | class ForecastingClient:
35 |     """HTTP-клиент для forecasting-service."""
36 |
37 |     def __init__(self):
38 |         self._base_url = settings.FORECASTING_SERVICE_URL.rstrip("/")

```



## src/main\_backend/external/integrations/forecasting\_client.py (continued)

```

39 |         self._timeout = settings.FORECASTING_CLIENT_TIMEOUT
40 |
41 |     @retry(
42 |         stop=stop_after_attempt(3),
43 |         wait=wait_exponential(multiplier=1, min=1, max=5),
44 |         retry=retry_if_exception_type((httpx.TimeoutException, httpx.ConnectError)),
45 |         reraise=True,
46 |     )
47 |     async def predict(self, features: dict, horizon: str = "now") -> ForecastingResult:
48 |         """Вызвать прогнозирование."""
49 |         url = f"{self._base_url}/api/v1/predictions/"
50 |         payload = {"features": features, "horizon": horizon}
51 |
52 |         try:
53 |             async with httpx.AsyncClient(timeout=self._timeout) as client:
54 |                 response = await client.post(url, json=payload)
55 |                 response.raise_for_status()
56 |                 body = response.json()
57 |
58 |                 return ForecastingResult(
59 |                     predicted_price=float(body["predicted_price"]),
60 |                     predicted_price_per_sqm=float(body["predicted_price_per_sqm"]),
61 |                     confidence=float(body.get("confidence", 0.85)),
62 |                     model_version=body.get("model_version", "unknown"),
63 |                 )
64 |
65 |         except httpx.ConnectError as e:
66 |             logger.error(f"Cannot connect to forecasting-service: {e}")
67 |             raise ForecastingServiceUnavailable(str(e))
68 |         except httpx.TimeoutException as e:
69 |             logger.error(f"Timeout calling forecasting-service: {e}")
70 |             raise ForecastingServiceUnavailable("timeout")
71 |         except httpx.HTTPStatusError as e:
72 |             logger.error(f"Forecasting-service HTTP error: {e.response.status_code}")
73 |             raise ForecastingServiceUnavailable(f"HTTP {e.response.status_code}")
74 |         except Exception as e:
75 |             logger.error(f"Unexpected error: {e}")
76 |             raise ForecastingServiceUnavailable(str(e))

```

## src/main\_backend/main.py

```

1 | # src/main_backend/main.py
2 | """
3 | FastAPI application factory (как в auth-service).
4 | """
5 | import logging
6 |
7 | from fastapi import FastAPI
8 | from fastapi.middleware.cors import CORSMiddleware
9 |
10 | from .api.responses import ApiResponse
11 | from .app.middleware.exception_handler import register_exception_handlers
12 | from .app.middleware.request_id import RequestIDMiddleware
13 | from .app.predictions.router import router as predictions_router
14 | from .config import settings
15 |
16 | logger = logging.getLogger(__name__)
17 |
18 |

```

## src/main\_backend/main.py (continued)

```
19 | def create_app() -> FastAPI:
20 |     """Фабрика FastAPI приложения."""
21 |     app = FastAPI(
22 |         title=settings.APP_NAME,
23 |         version=settings.APP_VERSION,
24 |         description="Real Estate Forecasting - Main Service",
25 |         docs_url="/docs" if settings.DEBUG else None,
26 |         redoc_url="/redoc" if settings.DEBUG else None,
27 |     )
28 |
29 |     # CORS
30 |     app.add_middleware(
31 |         CORSMiddleware,
32 |         allow_origins=settings.CORS_ORIGINS,
33 |         allow_credentials=settings.CORS_ALLOW_CREDENTIALS,
34 |         allow_methods=["*"],
35 |         allow_headers=["*"],
36 |     )
37 |
38 |     # Middleware
39 |     app.add_middleware(RequestIDMiddleware)
40 |
41 |     # Exception handlers
42 |     register_exception_handlers(app)
43 |
44 |     # Routers
45 |     app.include_router(predictions_router)
46 |
47 |     # Health check
48 |     @app.get("/health", tags=["Health"])
49 |     async def health_check():
50 |         return ApiResponse(data={"status": "ok", "service": "main-backend"})
51 |
52 |     logger.info(f" {settings.APP_NAME} v{settings.APP_VERSION} initialized")
53 |     return app
54 |
55 |
56 | app = create_app()
```